In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

To Do List App

AGENTS.md

Found in folder: **/**

This file provides project context and guidelines for developers working on the Todo app. It documents the technology stack, available commands, and coding conventions. It also establishes critical rules like extracting magic numbers into utility files.

AGENTS.md

This file gives Codex context about the project.

Project

A beginner-friendly Todo app built with React and Vite, used as a starter

project **for** a Codex course. Students extend the app by adding missing

features — guided by Codex.

Stack

| Layer | Choice |

|---|---|

| UI | React 19 |

| Bundler | Vite 7 |

| Styling | CSS Modules (plain CSS only) |

| Tests | Vitest + Testing Library |

| Persistence | localStorage |

No backend. No routing. No auth. No external UI libraries.

Commands

| Command | Purpose |

|---|---|

| `npm run dev` | Start dev server (localhost:5173)|

| `npm run build` | Production build |

| `npm test` | Run tests in watch mode |

| `npm run test:ui` | Open the Vitest browser UI |

Conventions

- All state lives in `App.jsx`; components receive data and callbacks as props

- One component per file; its CSS module lives in the same directory

- Task shape: `{ id: string, text: string, completed: boolean }`

- See `/docs/ui.md` for styling rules

- See `/docs/data-fetching.md` for state and persistence rules

Critical Rules

- Always extract magic numbers and configuration values into a dedicated

`src/utils/` file. Never hardcode them inline inside a component.

README.md

Found in folder: **/**

This file is a project README that documents a Todo app starter for a Codex course. It lists the app's existing features, missing features to be implemented as exercises, and provides setup instructions. The document also outlines the project structure and technology stack used.

Todo App — Codex Course Starter

A deliberately incomplete Todo app. You will use **Codex** to add the

missing features listed below.

Getting started

```bash

npm install

npm run dev # http://localhost:5173

```

Running tests

```bash
npm test         # watch mode
npm run test:ui    # browser UI
```

One test passes. One test fails — that is intentional. Fixing it is your first exercise.

What already works

Feature	File
Add a task	`TaskInput.jsx`
Mark complete (checkbox)	`TaskItem.jsx`
Delete a task	`TaskItem.jsx`
Character counter (max 100, turns red)	`TaskInput.jsx`
Filter bar — All / Active / Completed	`FilterBar.jsx`
localStorage persistence	`App.jsx`

What is missing (your exercises)

- **Empty-input validation** — clicking Add Task with a blank field still creates an empty task. Add a guard in `TaskInput.jsx` to prevent this. The failing test in `src/__tests__/tasks.test.jsx` will go green once you do.
- **Task editing** — double-clicking a task text should let you edit it in place and save on Enter / blur.
- **Due dates** — tasks have no deadline field. Add a date picker to `TaskInput` and display the date in `TaskItem`.
- **Drag to reorder** — tasks are fixed in insertion order.

Project structure

```
src/
  components/
    TaskInput.jsx + TaskInput.module.css
    TaskList.jsx  + TaskList.module.css
    TaskItem.jsx  + TaskItem.module.css
    FilterBar.jsx + FilterBar.module.css
  App.jsx + App.module.css
  main.jsx
  index.css
docs/
  ui.md          — styling conventions
  data-fetching.md — state and persistence conventions
vitest.config.js
```

Stack

- React 19 + Vite 7
- CSS Modules only **(**no Tailwind, no UI libraries**)**
- Vitest + Testing Library
- localStorage **(**no backend**)**

data-fetching.md

Found in folder: **/docs**

This documentation describes the state structure, persistence mechanism, and data flow for a todo application. It defines how task objects are shaped, stored in localStorage, and manipulated through callbacks. It also explains how tasks are filtered for display without mutating the original data.

Data & State Conventions

State Shape

All app state lives **in** `App.jsx`. There is no global state manager.

```js
// Task object — the single source of truth
{
  id: string,       // crypto.randomUUID(), set at creation time
  text: string,     // 1–100 characters (MAX_LENGTH enforced in TaskInput)
  completed: boolean // toggled by the checkbox
}
```

The active filter **(**`'all' **|** 'active' **|** 'completed'`**)** is also stored **in** `App`
but is never persisted — it resets to `'all'` on page load.

Persistence

Tasks are persisted to `localStorage` under the key `"todo-tasks"`.

Read — lazy initialiser so it runs exactly once on mount:

```js
const **[**tasks, setTasks**]** = useState**(()** **=>** **{**
  const stored = localStorage.getItem**(**'todo-tasks'**)**
  **return** stored ? JSON.parse**(**stored**)** **:** **[]**
**})**
```

Write — effect that syncs whenever `tasks` changes:

```js
useEffect**(()** **=>** **{**
  localStorage.setItem**(**'todo-tasks', JSON.stringify**(**tasks**))**
**}**, **[**tasks**])**
```

```

## Task Operations

All mutations live in `App.jsx` and are passed to children as callbacks:

| Prop | Signature | Behaviour |
|---|---|---|
| `onAdd` | `(text: string) => void` | Prepends a new task to the array |
| `onToggle` | `(id: string) => void` | Flips `completed` on one task |
| `onDelete` | `(id: string) => void` | Removes the task from the array |

Mutations always use the functional updater form (`setTasks(prev => ...)`) to
avoid stale-closure bugs.

## Filtering

Filtering is derived — it does **not** mutate `tasks`:

```js
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true
})
```

Only `filteredTasks` is passed to `<TaskList>`. The full `tasks` array is
used for the count in `<FilterBar>`.

What Is Not Here (yet)

- No API calls — everything is local; no `fetch`, no async state
- No loading or error states
- No optimistic updates
- No task editing, due dates, or ordering — those are student exercises in the Codex course
```

File extension: md

ui.md

Found in folder: **/docs**

This document defines UI conventions including layout structure, color palette, border radii, and interaction guidelines for consistent component styling. It also outlines the component hierarchy and provides steps for adding new components.

UI Conventions

Layout

- All components live **in** `src/components/`

- Each component has a matching `.module.css` file **in** the same directory

- Max content width: `600px`, centered via `margin: auto` **in** `App.module.css`

- Page background: `#f3f4f6` (set in `src/index.css`)

Colour Palette

| Token | Value | Used **for** |

|---|---|---|

| Primary | `#6366f1` | Buttons, focus rings, checkboxes |

| Primary dark | `#4f46e5` | Button hover |

| Danger | `#ef4444` | Delete hover, character limit |

| Danger light | `#fee2e2` | Delete button hover background |

| Border | `#d1d5db` | Default input / pill borders |

| Border inner | `#e5e7eb` | Task item border |

| Text muted | `#9ca3af` | Empty state, count, delete icon |

| Text base | `#1a1a1a` | Body copy |

| Surface | `#fff` | Cards, inputs |

Border Radii

| Element | Radius |

|---|---|

| Inputs, buttons | `6px` |

| Task list items | `8px` |

| Filter pills | `20px` |

| Delete button | `4px` |

Interaction

- Every interactive element has a `transition` on whichever property changes

- Default transition duration: `0.15s`

- Checkboxes use `accent-color: #6366f1`

Component Hierarchy

```

App

├── TaskInput — input field, Add Task button, character counter

├── FilterBar — All / Active / Completed pills + task count

└── TaskList

  └── TaskItem — checkbox, task text, delete button

```

Adding a New Component

1. Create `src/components/ComponentName.jsx`

2. Create `src/components/ComponentName.module.css` alongside it

3. Import CSS with `import styles from './ComponentName.module.css'`

4. Apply **class** names with `className={styles.className}`

5. Wire it into the appropriate parent component; lift any **new** state to `App.jsx`

App.jsx

Found in folder: **/src**

This code defines the main App component for a todo application. It manages tasks with state for the list, a filter, and editing, and persists tasks to local storage. It provides

functions to add, toggle, edit, and delete tasks, and renders input, filter, and list components.

```
import { useState, useEffect } from 'react'
import TaskInput from './components/TaskInput'
import TaskList from './components/TaskList'
import FilterBar from './components/FilterBar'
import styles from './App.module.css'
function App() {
 // Initialize tasks from localStorage, or start with an empty array
 const [tasks, setTasks] = useState(() => {
  const stored = localStorage.getItem('todo-tasks')
  return stored ? JSON.parse(stored) : []
 })
 const [filter, setFilter] = useState('all') // 'all', 'active', or 'completed'
 const [editingTaskId, setEditingTaskId] = useState(null) // ID of task being edited
 const [editingText, setEditingText] = useState('') // Text for the task being edited
 // Persist tasks to localStorage whenever the tasks state changes
 useEffect(() => {
  localStorage.setItem('todo-tasks', JSON.stringify(tasks))
 }, [tasks])
 const addTask = (text) => {
  setTasks(prev => [
   { id: crypto.randomUUID(), text, completed: false },
   ...prev, // Add new task to the beginning of the list
  ])
 }
```

```
const toggleTask = (id) => {
  setTasks(prev =>
    prev.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  )
}
const saveTaskEdit = () => {
  if (!editingTaskId) return
  const trimmedText = editingText.trim()
  if (trimmedText) {
    setTasks(prev =>
      prev.map(task =>
        task.id === editingTaskId ? { ...task, text: trimmedText } : task
      )
    )
  }
  setEditingTaskId(null)
  setEditingText('')
}
const startTaskEdit = (task) => {
  // If editing a different task, save it first
  if (editingTaskId && editingTaskId !== task.id) {
    saveTaskEdit()
  }
  setEditingTaskId(task.id)
```

```
  setEditingText(task.text)
}
const cancelTaskEdit = () => {
  setEditingTaskId(null)
  setEditingText('')
}
const deleteTask = (id) => {
  // If deleting the task currently being edited, cancel edit first
  if (editingTaskId === id) {
    cancelTaskEdit()
  }
  setTasks(prev => prev.filter(task => task.id !== id))
}
// Filter tasks based on the current filter state
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true // 'all' filter
})
return (
  <div className={styles.app}>
    <h1 className={styles.title}>Todo</h1>
    <TaskInput onAdd={addTask} />
    <FilterBar filter={filter} onFilterChange={setFilter} taskCount={tasks.length} />
    <TaskList
      tasks={filteredTasks}
```

```
      editingTaskId={editingTaskId}
      editingText={editingText}
      onEditStart={startTaskEdit}
      onEditTextChange={setEditingText}
      onEditSave={saveTaskEdit}
      onEditCancel={cancelTaskEdit}
      onToggle={toggleTask}
      onDelete={deleteTask}
    />
  </div>
 )
}
export default App
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component inside a React StrictMode wrapper into the HTML element with the ID ‘root’. It imports necessary React modules, global styles, and the root App component. The application is mounted to the DOM element identified as ‘root’.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css' // Global styles
import App from './App.jsx'
// Mount the React app to the DOM element with ID 'root'
createRoot(document.getElementById('root')).render(
 <StrictMode>
  <App />
```

```
  </StrictMode>
)
```

FilterBar.jsx

Found in folder: **/src/components**

This code renders a set of filter buttons and a task counter. It highlights the currently active filter and updates the filter when a button is clicked. The task counter displays the number of tasks with proper pluralization.

```
import styles from './FilterBar.module.css'

// Available filter options
const FILTERS = ['all', 'active', 'completed']

export default function FilterBar({ filter, onFilterChange, taskCount }) {
  return (
    <div className={styles.bar}>
      {FILTERS.map(f => (
        <button
          key={f}
          className={`${styles.filterButton} ${filter === f ? styles.active : ''}`}
          onClick={() => onFilterChange(f)}
        >
          {f.charAt(0).toUpperCase() + f.slice(1)} {/* Capitalize filter name */}
        </button>
      ))}
      <span className={styles.count}>
        {taskCount} {taskCount === 1 ? 'task' : 'tasks'} {/* Handle pluralization */}
      </span>
    </div>
  )
```

}

TaskInput.jsx

Found in folder: **/src/components**

This component provides a text input and button for adding new tasks. It tracks the input text, trims whitespace when adding, and enforces a maximum length. It also displays a character counter that changes style when the limit is reached.

```
import { useState } from 'react'
import { TASK_TEXT_MAX_LENGTH } from '../utils/taskConfig'
import styles from './TaskInput.module.css'
export default function TaskInput({ onAdd }) {
 const [text, setText] = useState('')
 const handleAdd = () => {
  // TODO: add empty-input validation here — currently submits blank tasks
  onAdd(text.trim())  // Pass trimmed text to parent handler
  setText('')       // Clear input after adding
 }
 return (
  <div className={styles.wrapper}>
   <div className={styles.inputRow}>
    <input
     className={styles.input}
     type="text"
     value={text}
     onChange={e => setText(e.target.value)}
     onKeyDown={e => e.key === 'Enter' && handleAdd()}  // Submit on Enter key
     placeholder="What needs to be done?"
     maxLength={TASK_TEXT_MAX_LENGTH}
```

```
        aria-label="New task"
      />
      <button className={styles.button} onClick={handleAdd}>
        Add Task
      </button>
    </div>
    {/* Character counter with visual limit indicator */}
    <p className={`${styles.counter} ${text.length >= TASK_TEXT_MAX_LENGTH ? styles.atLimit : ''}`}>
      {text.length} / {TASK_TEXT_MAX_LENGTH}
    </p>
  </div>
 )
}
```

TaskItem.jsx

Found in folder: **/src/components**

This component renders a single task item in a list, displaying its text and allowing it to be toggled, edited, or deleted. It manages an editing state where the task text becomes a focused input field, handling save and cancel actions via keyboard and blur events. It also plays a sound when a task is marked as complete.

```
import { useEffect, useRef } from 'react'
import { playCompletionSound } from '../utils/audio'
import { TASK_TEXT_MAX_LENGTH } from '../utils/taskConfig'
import styles from './TaskItem.module.css'
export default function TaskItem({
 task,
 isEditing,
```

```
  editingText,
  onEditStart,
  onEditTextChange,
  onEditSave,
  onEditCancel,
  onToggle,
  onDelete,
}) {
  const editInputRef = useRef(null)
  const isCancellingEditRef = useRef(false) // Tracks if edit was cancelled via Escape key to prevent onBlur from saving
  // Focus and select text in the input field when editing mode starts
  useEffect(() => {
    if (isEditing) {
      editInputRef.current?.focus()
      editInputRef.current?.select()
    }
  }, [isEditing])
  const handleToggle = () => {
    if (!task.completed) {
      playCompletionSound() // Play sound only when marking as complete, not when unchecking
    }
    onToggle(task.id)
  }
  const handleEditKeyDown = (event) => {
    if (event.key === 'Enter') {
```

```
    onEditSave() // Save on Enter key
  }
  if (event.key === 'Escape') {
    isCancellingEditRef.current = true // Set flag to indicate cancellation
    onEditCancel()
  }
}
const handleEditBlur = () => {
  // If edit was cancelled via Escape, don't save on blur
  if (isCancellingEditRef.current) {
    isCancellingEditRef.current = false
    return
  }
  onEditSave() // Save on blur (when input loses focus)
}
return (
  <li className={`${styles.item} ${task.completed ? styles.completed : ''}`}>
    <input
      type="checkbox"
      className={styles.checkbox}
      checked={task.completed}
      onChange={handleToggle}
      aria-label={`Mark "${task.text}" as ${task.completed ? 'active' : 'complete'}`}
    />
    {isEditing ? (
      <input
```

```
        ref={editInputRef}
        className={styles.editInput}
        type="text"
        value={editingText}
        onChange={event => onEditTextChange(event.target.value)}
        onKeyDown={handleEditKeyDown}
        onBlur={handleEditBlur}
        maxLength={TASK_TEXT_MAX_LENGTH}
        aria-label={`Edit "${task.text}"`}
      />
    ) : (
      <span className={styles.text} onDoubleClick={() => onEditStart(task)}>
        {task.text}
      </span>
    )}
    <button
      className={styles.deleteButton}
      onClick={() => onDelete(task.id)}
      aria-label={`Delete "${task.text}"`}
    >
      ×
    </button>
  </li>
 )
}
```

TaskList.jsx

Found in folder: **/src/components**

This component renders a list of tasks. If the list is empty, it displays a message. Otherwise, it maps each task to a TaskItem component, passing down the task data and all relevant event handlers.

```
import TaskItem from './TaskItem'

import styles from './TaskList.module.css'

export default function TaskList({
  tasks,
  editingTaskId,
  editingText,
  onEditStart,
  onEditTextChange,
  onEditSave,
  onEditCancel,
  onToggle,
  onDelete,
}) {
  // Show a placeholder message if there are no tasks
  if (tasks.length === 0) {
    return <p className={styles.empty}>No tasks here.</p>
  }
  return (
    <ul className={styles.list}>
      {tasks.map(task => (
        <TaskItem
          key={task.id}
          task={task}
```

```
        isEditing={editingTaskId === task.id} // Determine if this specific task is being edited
        editingText={editingText}
        onEditStart={onEditStart}
        onEditTextChange={onEditTextChange}
        onEditSave={onEditSave}
        onEditCancel={onEditCancel}
        onToggle={onToggle}
        onDelete={onDelete}
      />
    ))}
  </ul>
 )
}
```

TaskNote.jsx

Found in folder: **/src/components**

This component conditionally renders a note for a task if one exists, providing an accessible label. It uses CSS modules for styling and returns a paragraph element containing the note text.

```
import styles from './TaskNote.module.css'

export default function TaskNote({ task }) {
 // Do not render anything if the task has no note
 if (!task.note) return null
 return (
  // Provide an accessible description for the note
  <p className={styles.item} aria-label={`Note for "${task.text}"`}>
   <span className={styles.text}>{task.note}</span>
  </p>
```

```
  )
}
```

tasks.test.jsx

Found in folder: **/src/__tests__**

This code tests a Todo app's UI behavior, including adding, editing, and completing tasks. It verifies that tasks are properly managed and that specific actions like double-clicking to edit or pressing Escape to cancel work correctly. It also includes a test for a sound effect when marking a task complete, using a mocked audio context.

```
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, beforeEach, vi } from 'vitest'
import App from '../App'

// Helper function to add a task via UI interactions
const addTask = (text) => {
  fireEvent.change(screen.getByLabelText('New task'), {
    target: { value: text },
  })
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
}

// Reset state before each test to ensure isolation
beforeEach(() => {
  localStorage.clear()
  vi.restoreAllMocks()
  delete window.AudioContext
  delete window.webkitAudioContext
})

describe('Todo App', () => {
  // PASSING: the app should render with a visible heading.
```

```
it('renders the app title', () => {
  render(<App />)
  expect(screen.getByRole('heading', { name: 'Todo' })).toBeInTheDocument()
})
// FAILING: this test documents a missing feature.
//
// The app currently adds a task even when the input is empty.
// Your job: open TaskInput.jsx and add a guard in handleAdd() so that
// clicking "Add Task" with a blank field does nothing.
// Once you do, this test will go green.
it('does not add a task when the input is empty', () => {
  render(<App />)
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
  const items = screen.queryAllByRole('listitem')
  expect(items).toHaveLength(0)
})
// Tests that a sound is played when a task is marked complete
it('plays a short sound when a task is marked complete', () => {
  const start = vi.fn()
  const stop = vi.fn()
  const connect = vi.fn()
  const setValueAtTime = vi.fn()
  const exponentialRampToValueAtTime = vi.fn()
  // Mock AudioContext to avoid real audio and track calls
  class FakeAudioContext {
    constructor() {
```

```
    this.currentTime = 0
    this.destination = {}
    this.state = 'running'
  }
  createOscillator() {
    return {
      connect,
      frequency: { setValueAtTime, exponentialRampToValueAtTime },
      start,
      stop,
      type: '',
    }
  }
  createGain() {
    return {
      connect,
      gain: { setValueAtTime, exponentialRampToValueAtTime },
    }
  }
}
window.AudioContext = FakeAudioContext
render(<App />)
fireEvent.change(screen.getByLabelText('New task'), {
  target: { value: 'Write sound test' },
})
fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
```

```
  const checkbox = screen.getByRole('checkbox', {
    name: 'Mark "Write sound test" as complete',
  })
  fireEvent.click(checkbox)
  expect(start).toHaveBeenCalledTimes(1)
  expect(stop).toHaveBeenCalledTimes(1)
  // Sound should only play when marking complete, not when unchecking
  fireEvent.click(checkbox)
  expect(start).toHaveBeenCalledTimes(1)
})
it('shows an edit input when a task is double-clicked', () => {
  render(<App />)
  addTask('Edit this task')
  fireEvent.doubleClick(screen.getByText('Edit this task'))
  expect(screen.getByRole('textbox', { name: 'Edit "Edit this task"' })).toHaveValue(
    'Edit this task',
  )
})
it('saves edited task text when Enter is pressed', () => {
  render(<App />)
  addTask('Old task text')
  fireEvent.doubleClick(screen.getByText('Old task text'))
  const editInput = screen.getByRole('textbox', { name: 'Edit "Old task text"' })
  fireEvent.change(editInput, { target: { value: 'New task text' } })
  fireEvent.keyDown(editInput, { key: 'Enter' })
  expect(screen.getByText('New task text')).toBeInTheDocument()
```

```
  expect(screen.queryByText('Old task text')).not.toBeInTheDocument()
})
it('saves edited task text when the edit input loses focus', () => {
  render(<App />)
  addTask('Blur save task')
  fireEvent.doubleClick(screen.getByText('Blur save task'))
  const editInput = screen.getByRole('textbox', { name: 'Edit "Blur save task"' })
  fireEvent.change(editInput, { target: { value: 'Saved on blur' } })
  fireEvent.blur(editInput)
  expect(screen.getByText('Saved on blur')).toBeInTheDocument()
  expect(screen.queryByText('Blur save task')).not.toBeInTheDocument()
})
it('cancels edited task text when Escape is pressed', () => {
  render(<App />)
  addTask('Keep original text')
  fireEvent.doubleClick(screen.getByText('Keep original text'))
  const editInput = screen.getByRole('textbox', { name: 'Edit "Keep original text"' })
  fireEvent.change(editInput, { target: { value: 'Discard this text' } })
  fireEvent.keyDown(editInput, { key: 'Escape' })
  expect(screen.getByText('Keep original text')).toBeInTheDocument()
  expect(screen.queryByText('Discard this text')).not.toBeInTheDocument()
})
// Ensures empty or whitespace-only edits are ignored
it('keeps the original task text when a blank edit is saved', () => {
  render(<App />)
  addTask('Do not blank me')
```

```
    fireEvent.doubleClick(screen.getByText('Do not blank me'))
    const editInput = screen.getByRole('textbox', { name: 'Edit "Do not blank me"' })
    fireEvent.change(editInput, { target: { value: '   ' } })
    fireEvent.keyDown(editInput, { key: 'Enter' })
    expect(screen.getByText('Do not blank me')).toBeInTheDocument()
  })
})
```

index.html

Found in folder: **/**

This code defines the basic structure of an HTML page, setting up metadata, linking resources, and providing a container for a React application to render into.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <!-- Root element where the React application will be mounted -->
    <div id="root"></div>
    <!-- Entry point script for the Vite-powered React application -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

App.module.css

Found in folder: **/src**

This CSS code defines styles for two classes: `.app` centers a container with a maximum width and sets a system font, while `.title` styles a large, bold, centered heading with specific spacing and color.

```
.app {
  max-width: 600px;        /* Limits container width */
  margin: 48px auto;       /* Centers container with top/bottom margin */
  padding: 0 16px;         /* Adds horizontal padding */
  font-family: system-ui, -apple-system, sans-serif; /* Uses system fonts */
}
.title {
  font-size: 2rem;         /* Sets large font size */
  font-weight: 700;        /* Makes text bold */
  text-align: center;      /* Centers text horizontally */
  margin: 0 0 32px;        /* Adds bottom margin only */
  color: #1a1a1a;          /* Sets dark gray color */
  letter-spacing: -0.5px;  /* Tightens letter spacing slightly */
}
```

index.css

Found in folder: **/src**

This CSS code sets global styling rules for all elements and the body. It applies the border-box box-sizing model to every element and its pseudo-elements. It also removes default body margins, sets a light gray background with dark text, and enables font smoothing for better readability on WebKit browsers.

```
*,
*::before,
```

```
*::after {
  box-sizing: border-box; /* Includes padding and border in element's total width/height */
}
body {
  margin: 0; /* Remove default browser margin */
  background: #f3f4f6; /* Light gray background */
  color: #1a1a1a; /* Dark text color for contrast */
  -webkit-font-smoothing: antialiased; /* Improves font rendering on WebKit browsers */
}
```

FilterBar.module.css

Found in folder: **/src/components**

This CSS defines styles for a filter bar component, including layout for the bar itself and interactive buttons for filtering. The filter buttons change appearance on hover and when active to provide visual feedback. A count element is styled to align to the right with subdued coloring.

```
.bar {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
  flex-wrap: wrap;
}
.filterButton {
  padding: 6px 14px;
  font-size: 0.875rem;
  border: 2px solid #d1d5db;
  border-radius: 20px;
```

```
  background: #fff;
  color: #374151;
  cursor: pointer;
  transition: all 0.15s; /* Smooth transition for hover/active states */
}
.filterButton:hover {
  border-color: #6366f1;
  color: #6366f1;
}
.filterButton.active {
  background: #6366f1;
  border-color: #6366f1;
  color: #fff;
}
.count {
  margin-left: auto; /* Pushes count to the right side of the flex container */
  font-size: 0.875rem;
  color: #9ca3af;
}
```

TaskInput.module.css

Found in folder: **/src/components**

This CSS file defines styles for a form component with an input field, a button, and a character counter. It styles the layout, appearance, and interactive states like focus and hover. The counter text changes color when a limit is reached.

```
.wrapper {
  margin-bottom: 16px;
}
```

```
.inputRow {
  display: flex;
  gap: 8px; /* Space between input and button */
}
.input {
  flex: 1; /* Input expands to fill available space */
  padding: 10px 14px;
  font-size: 1rem;
  border: 2px solid #d1d5db;
  border-radius: 6px;
  outline: none;
  background: #fff;
  transition: border-color 0.15s;
}
.input:focus {
  border-color: #6366f1; /* Highlight border on focus */
}
.button {
  padding: 10px 18px;
  font-size: 1rem;
  font-weight: 600;
  background: #6366f1;
  color: #fff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
```

```
  white-space: nowrap; /* Prevent button text from wrapping */
  transition: background 0.15s;
}
.button:hover {
  background: #4f46e5; /* Darker shade on hover */
}
.counter {
  margin: 4px 0 0;
  font-size: 0.8rem;
  color: #6b7280;
  text-align: right;
}
.atLimit {
  color: #ef4444; /* Red text to indicate limit reached */
  font-weight: 600;
}
```

TaskItem.module.css

Found in folder: **/src/components**

This CSS styles a list item component with a checkbox, text, and delete button. It defines visual states for hover, completion, and editing. The layout uses flexbox for alignment and spacing.

```
.item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 14px;
  background: #fff;
```

```
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  transition: background 0.1s;
}
.item:hover {
  background: #f9fafb; /* Subtle hover feedback */
}
.checkbox {
  width: 18px;
  height: 18px;
  flex-shrink: 0; /* Prevent checkbox from shrinking */
  cursor: pointer;
  accent-color: #6366f1; /* Custom color for the checkbox */
}
.text {
  flex: 1; /* Take up remaining space */
  font-size: 0.95rem;
  color: #1a1a1a;
  word-break: break-word; /* Prevent text overflow */
}
.completed .text {
  text-decoration: line-through;
  color: #9ca3af; /* Muted color for completed items */
}
.editInput {
  flex: 1;
```

```
  min-width: 0; /* Allows input to shrink below its default minimum */
  padding: 6px 8px;
  font-size: 0.95rem;
  color: #1a1a1a;
  background: #fff;
  border: 2px solid #6366f1; /* Highlighted border for edit state */
  border-radius: 6px;
  outline: none;
}
.deleteButton {
  flex-shrink: 0;
  padding: 4px 8px;
  font-size: 0.75rem;
  background: transparent;
  border: 1px solid transparent;
  border-radius: 4px;
  cursor: pointer;
  color: #9ca3af;
  line-height: 1;
  transition: all 0.15s;
}
.deleteButton:hover {
  background: #fee2e2;
  border-color: #fca5a5;
  color: #ef4444; /* Red color scheme on hover */
}
```

TaskList.module.css

Found in folder: **/src/components**

This CSS defines two classes: `.list` styles an unordered list as a vertical flex container with no default styling and spacing between items, while `.empty` styles an element for displaying empty states with centered, muted text and vertical padding.

```
.list {
  list-style: none;   /* Remove default bullet points */
  margin: 0;        /* Remove default margin */
  padding: 0;       /* Remove default padding */
  display: flex;     /* Use flexbox layout */
  flex-direction: column; /* Stack children vertically */
  gap: 8px;         /* Space between child elements */
}
.empty {
  text-align: center; /* Center text horizontally */
  color: #9ca3af;     /* Set a muted gray color */
  font-size: 0.9rem;  /* Use a slightly smaller font size */
  padding: 40px 0;    /* Add vertical padding */
  margin: 0;        /* Remove default margin */
}
```

TaskNote.module.css

Found in folder: **/src/components**

This CSS code defines two classes: `.item` creates a flex container with padding, border, and rounded corners for list items, while `.text` styles text elements to take up available space with specific font size and color.

```
.item {
  display: flex;            /* Use flexbox layout */
```

```
  align-items: center;        /* Vertically center child elements */
  gap: 12px;                  /* Space between child elements */
  padding: 8px 14px;          /* Internal spacing */
  margin: 0;                  /* Remove default margins */
  background: #fff;           /* White background */
  border: 1px solid #e5e7eb;  /* Light gray border */
  border-radius: 8px;         /* Rounded corners */
}
.text {
  flex: 1;                    /* Take up remaining space in flex container */
  font-size: 0.85rem;         /* Smaller font size */
  color: #6b7280;             /* Medium gray text color */
  word-break: break-word;     /* Break long words to prevent overflow */
}
```

audio.js

Found in folder: **/src/utils**

This code defines a function that plays a short rising sine wave sound when called. It creates an audio oscillator that sweeps from a lower to a higher frequency while fading in and out in volume. The function handles browser audio context initialization and gracefully fails if audio playback is blocked.

```
const COMPLETION_START_FREQUENCY = 660
const COMPLETION_END_FREQUENCY = 880
const COMPLETION_RAMP_DURATION = 0.08
const COMPLETION_DURATION = 0.18
const COMPLETION_START_VOLUME = 0.001
const COMPLETION_PEAK_VOLUME = 0.08
const COMPLETION_ATTACK_DURATION = 0.01
```

```
let completionAudioContext

export function playCompletionSound() {
  // Exit if not in browser environment
  if (typeof window === 'undefined') return
  const AudioContext = window.AudioContext || window.webkitAudioContext
  if (!AudioContext) return
  try {
    // Reuse audio context if available, otherwise create new one
    const audioContext = completionAudioContext || new AudioContext()
    completionAudioContext = audioContext
    if (audioContext.state === 'suspended') {
      audioContext.resume()
    }
    const oscillator = audioContext.createOscillator()
    const gain = audioContext.createGain()
    const startTime = audioContext.currentTime
    oscillator.type = 'sine'
    oscillator.frequency.setValueAtTime(COMPLETION_START_FREQUENCY, startTime)
    oscillator.frequency.exponentialRampToValueAtTime(
      COMPLETION_END_FREQUENCY,
      startTime + COMPLETION_RAMP_DURATION,
    )
    // Create volume envelope: fade in quickly, then fade out
    gain.gain.setValueAtTime(COMPLETION_START_VOLUME, startTime)
    gain.gain.exponentialRampToValueAtTime(
      COMPLETION_PEAK_VOLUME,
```

```
    startTime + COMPLETION_ATTACK_DURATION,
  )
  gain.gain.exponentialRampToValueAtTime(
    COMPLETION_START_VOLUME,
    startTime + COMPLETION_DURATION,
  )
  oscillator.connect(gain)
  gain.connect(audioContext.destination)
  oscillator.start(startTime)
  oscillator.stop(startTime + COMPLETION_DURATION)
 } catch {
  // Audio playback can be blocked by browser policy or unavailable in tests.
 }
}
```

taskConfig.js

Found in folder: **/src/utils**

This code defines a constant for the maximum allowed length of task text. It exports this constant so it can be imported and used in other files. This ensures a consistent text length limit is enforced across the application.

```
// Maximum allowed character length for task text
export const TASK_TEXT_MAX_LENGTH = 100
```

Daily Reader App

AGENTS.md

Found in folder: **/**

This file provides documentation for a React + Vite article showcase project used as a coding-course starter. It outlines the project's technology stack, available commands, and

development conventions. It also lists critical rules that must be followed, such as avoiding routing and external UI libraries.

AGENTS.md

Project

A React + Vite article showcase site built as a coding-course starter project. Students work through exercises by completing missing features **in this** codebase.

Stack

- **React 19** — functional components, hooks

- **Vite 7** — dev server and bundler

- **CSS Modules** — scoped component styles, no external UI libraries

Commands

```bash

npm run dev      # start dev server at http://localhost:5173

npm run build    # production build

npm run preview  # preview production build

npm run lint     # run ESLint

```

Conventions

- One component per file; component file and its CSS Module share the same base name.

- Props flow down only — no context, no global state store.

- State lives **in** `App.jsx`; components receive data and callbacks via props.

- CSS **class** names use camelCase **in** JS **(**`styles.backButton`**)**.

- Article data lives **in** `src/data/articles.js` as a plain JS array; no API calls.

Critical Rules

- **Do** not add routing.** Navigation between list and detail views is managed by a single `selectedArticle` state **in** `App.jsx`. Introducing React Router or any router breaks the intended teaching structure.

- **Do not add external UI libraries.** No Tailwind, no MUI, no Chakra. Only CSS Modules and plain CSS.

- **Do not add a backend or localStorage.** All data is static and lives in `src/data/articles.js`.

- **Missing features may be intentional.** This is a course starter — gaps in the code are often student exercises. Do not silently add features without being explicitly asked.

- **Keep dependencies minimal.** Only `react`, `react-dom`, and Vite-related devDependencies are permitted.

README.md

Found in folder: **/**

This file contains documentation explaining the setup and features of a React project using Vite, including available plugins, React Compiler details, and ESLint expansion recommendations.

React + Vite

This template provides a minimal setup to get React working in Vite with HMR and some ESLint rules.

Currently, two official plugins are available:

- [@vitejs/plugin-react](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react) uses [Babel](https://babeljs.io/) (or [oxc](https://oxc.rs) when used in [rolldown-vite](https://vite.dev/guide/rolldown)) for Fast Refresh

- [@vitejs/plugin-react-swc](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react-swc) uses [SWC](https://swc.rs/) for Fast Refresh

React Compiler

The React Compiler is enabled on this template. See [this documentation](https://react.dev/learn/react-compiler) for more information.

Note: This will impact Vite dev & build performances.

Expanding the ESLint configuration

If you are developing a production application, we recommend using TypeScript with type-aware lint rules enabled. Check out the [TS

template**](**https://github.com/vitejs/vite/tree/main/packages/create-vite/template-react-ts) for information on how to integrate TypeScript and [`typescript-eslint`](https://typescript-eslint.io) in your project.

SKILL.md

Found in folder: **/.codex/skills/add-article**

This code defines a GitHub Actions workflow that automates the process of researching and adding a new article to a React/Vite project's data file. It provides step-by-step instructions for inspecting the existing data structure, researching a topic, drafting content, and validating the changes. The workflow ensures the new article matches the project's formatting and style guidelines before appending it to the file.

---

name: add-article

description: Research a requested topic, write a 200-250 word general-audience article, and add it to **this** React/Vite course project's src/data/articles.js file using the existing article data structure. Use when the user asks to add, create, research, or write a new article entry for the static article showcase data file.

Add Article

Workflow

1. Inspect `src/data/articles.js` before editing.

  - Preserve the existing exported data structure, property order, indentation, quote style, and template-literal content style.

  - Choose the next numeric `id` after the highest existing article id.

  - Use `author: "Editorial Team"` unless the user explicitly provides another author.

  - Use today's date from the runtime context **in** `YYYY-MM-DD` format unless the user provides a different date.

2. Research the requested topic before drafting.

  - Use reliable sources appropriate to the topic, preferring primary or authoritative sources.

- **For** current, technical, scientific, medical, legal, or financial topics, verify facts with up-to-date sources.

- Keep notes source-grounded, but **do** not add citations inside the article body unless the existing article style already does.

3. Draft the article.

- Write **for** a general audience.

- Keep `content` between 200 and 250 words.

- Use accurate, cautious wording where experts still debate details.

- Provide a concise `title` and `summary` matching the style **of** the existing entries.

- Avoid unsupported claims, jargon-heavy explanations, and promotional tone.

4. Edit `src/data/articles.js`.

- Append one **new** object to the existing array.

- **Do** not add routing, API calls, localStorage, **new** dependencies, or unrelated features.

- **Do** not rewrite existing article content unless the user explicitly asks.

5. Validate the change.

- Count only the words **in** the **new** `content` field and confirm it is 200-250 words.

- Run `npm run lint` when available.

- **In** the final response, mention the added file, article word count, lint result, and sources used **for** research.

Implementation Notes

- Use `apply_patch` **for** manual edits.

- **If** shell quoting makes word counting against template literals brittle, rerun with safer quoting or inspect the object another way.

- Keep the final response brief and focused on what changed and what was verified.

content.md

Found in folder: **/docs**

This file provides documentation for managing articles in the data file. It explains the structure of article objects and gives step-by-step instructions for adding or removing articles. It also includes content guidelines for writing summaries and full articles.

Content Guide

Article data structure

All articles live **in** `src/data/articles.js` as elements **of** a plain JavaScript array exported as the default export.

Each article object has the following shape:

```js
{
  id: 1,              // number — unique identifier; increment for each new article
  title: "...",       // string — displayed on the card and as the article heading
  summary: "...",     // string — one sentence; shown on the card only
  content: `...`,     // string — full article text; shown in the detail view
  author: "Editorial Team", // string — displayed on card and detail view
  date: "2025-11-15",   // string — ISO 8601 date (YYYY-MM-DD)
}
```

How to add a new article

1. Open `src/data/articles.js`.
2. Copy the shape above and fill **in** all fields.
3. Give it the next available `id` **(**increment the highest existing id by 1**)**.
4. Add the object to the `articles` array.
5. Save the file — the dev server will hot-reload and the **new** card will appear on the home page.

No imports, no API calls, no database changes needed.

Content guidelines

- **summary**: One sentence, ≤ 160 characters. Should make a reader want to click.

- **content**: Plain prose paragraphs separated by a blank line (`\n\n`). The detail view splits on `\n\n` to render each paragraph as its own `<p>` tag. Aim for 200–300 words.

- **date**: Use a realistic date in `YYYY-MM-DD` format. The date is displayed as-is; no formatting is applied.

- **author**: Currently always `"Editorial Team"`. If per-author names are added later, update the card and detail templates accordingly.

Removing an article

Delete the object from the array. The list re-renders automatically. Ids only need to be unique, not contiguous.

ui.md

Found in folder: **/docs**

This markdown file documents UI conventions and styling guidelines for a project using CSS Modules. It defines color tokens, typography rules, layout specifications, and interactive states. The document also establishes naming conventions and explains the styling approach.

UI Conventions

Styling approach

All component styles use **CSS Modules** (`.module.css` files). Class names are imported as a `styles` object and applied via `className={styles.myClass}`. There are no global utility classes and no external UI frameworks.

A minimal global reset lives in `src/index.css` and is imported once in `src/main.jsx`. It sets `box-sizing: border-box`, resets `body` margin, and defines the base font stack. Do not add component-specific rules here.

Colour palette

| Token | Value | Usage |

|---|---|---|

| Background | `#f4f5f7` | Page background (`body`) |

| Surface | `#ffffff` | Cards, article containers |

| Border | `#e5e7eb` | Card borders, dividers |

| Text primary | `#1a1a2e` | Headings |

| Text body | `#374151` | Article body copy |

| Text muted | `#4b5563` — `#9ca3af` | Summaries, meta lines |

| Accent | `#2563eb` | Buttons, site name |

| Accent hover | `#1d4ed8` | Button hover state |

Typography

- **Font stack**: `system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif`

- **Line height**: `1.6` for body text; `1.75` inside article content paragraphs

- **Heading weights**: `800` for page/article headings; `700` for card titles

Layout

- Page content is centred with a `max-width` on the container: `1000px` for the list view, `720px` for the detail view.

- Cards are laid out in a responsive CSS Grid: `repeat(auto-fill, minmax(300px, 1fr))` — no media queries required.

- Cards use `flex-direction: column` internally so the "Read article" button always sits at the bottom.

Interactive states

- Cards: `transform: translateY(-3px)` + increased box-shadow on `:hover`.

- Buttons: background colour darkens on `:hover` via a CSS `transition`.

- All transitions use `ease` timing at `0.15s`–`0.18s`.

Naming

CSS Module **class** names use camelCase (e.g., `.backButton`, `.siteHeader`) so they map cleanly to JS property access.

App.jsx

Found in folder: **/src**

This code renders a news reading app with a header and main content area. It displays either a list of articles or a detailed view of a selected article. The user can click an article from the list to see its details and then return to the list.

```
import { useState } from 'react';
import ArticleList from './components/ArticleList';
import ArticleDetail from './components/ArticleDetail';
import articles from './data/articles';
import styles from './App.module.css';
const App = () => {
 // Tracks which article, if any, is currently selected for detailed view
 const [selectedArticle, setSelectedArticle] = useState(null);
 return (
  <div className={styles.app}>
   <header className={styles.siteHeader}>
    <span className={styles.siteName}>The Daily Read</span>
   </header>
   <main>
    {/* Conditionally render the article detail view or the list */}
    {selectedArticle ? (
     <ArticleDetail
      article={selectedArticle}
      onBack={() => setSelectedArticle(null)} // Returns to list view
     />
    ) : (
     <ArticleList articles={articles} onSelect={setSelectedArticle} />
    )}
   </main>
```

```
  </div>
 );
};
export default App;
```

main.jsx

Found in folder: **/src**

This code initializes a React application by creating a root rendering container and mounting the main App component inside React's StrictMode wrapper. It attaches the application to the HTML element with the ID 'root' and applies global CSS styles. StrictMode enables additional development checks for potential problems in the application.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css'; // Global styles
import App from './App.jsx';
// Create root rendering container targeting the 'root' DOM element
const root = createRoot(document.getElementById('root'));
// Render the App component wrapped in StrictMode for development checks
root.render(
 <StrictMode>
  <App />
 </StrictMode>
);
```

ArticleCard.jsx

Found in folder: **/src/components**

This component renders a clickable article card displaying its title, author, date, and summary. It triggers a selection function when clicked anywhere on the card or on its

dedicated button. The card uses CSS modules for styling and includes an accessible label for the button.

```
import styles from './ArticleCard.module.css';

const ArticleCard = ({ article, onSelect }) => {
  return (
    <article className={styles.card} onClick={() => onSelect(article)}>
      <h2 className={styles.title}>{article.title}</h2>
      <p className={styles.meta}>
        {article.author} · {article.date}
      </p>
      <p className={styles.summary}>{article.summary}</p>
      {/* Button also triggers onSelect via parent click, but provides explicit semantic action */}
      <button className={styles.button} aria-label={`Read ${article.title}`}>
        Read article
      </button>
    </article>
  );
};

export default ArticleCard;
```

ArticleDetail.jsx

Found in folder: **/src/components**

This component displays a detailed article view with a back button, article title, metadata, and content split into paragraphs. It receives an article object and an onBack handler as props to render the article details and handle navigation. The content is formatted by splitting the article content on double newlines to create separate paragraph elements.

```
import styles from './ArticleDetail.module.css';
```

```
const ArticleDetail = ({ article, onBack }) => {
  return (
    <div className={styles.container}>
      {/* Back button triggers the provided onBack handler */}
      <button className={styles.backButton} onClick={onBack}>
        ← Back to articles
      </button>
      <article className={styles.article}>
        <h1 className={styles.title}>{article.title}</h1>
        <p className={styles.meta}>
          {article.author} · {article.date}
        </p>
        <div className={styles.content}>
          {/* Split content by double newlines to render separate paragraphs */}
          {article.content.split('\n\n').map((paragraph, index) => (
            <p key={index}>{paragraph}</p>
          ))}
        </div>
      </article>
    </div>
  );
};
export default ArticleDetail;
```

ArticleList.jsx

Found in folder: **/src/components**

This code renders a list of articles, displaying a header with a count and a grid of individual article cards. It uses a functional component that takes an array of articles and a selection handler as props. Each article is passed to a reusable `ArticleCard` component for display.

```
import ArticleCard from './ArticleCard';

import styles from './ArticleList.module.css';

// Component to display a list of articles with a header and a grid layout

const ArticleList = ({ articles, onSelect }) => {

 return (

  <section className={styles.section}>

   <header className={styles.header}>

    <h1 className={styles.heading}>Latest Articles</h1>

    {/* Dynamically pluralize 'article' based on the count */}

    <p className={styles.subheading}>

     {articles.length} article{articles.length !== 1 ? 's' : ''}

    </p>

   </header>

   <div className={styles.grid}>

    {/* Render an ArticleCard for each article in the list */}

    {articles.map((article) => (

     <ArticleCard key={article.id} article={article} onSelect={onSelect} />

    ))}

   </div>

  </section>

 );

};

export default ArticleList;
```

index.html

Found in folder: **/**

This HTML file defines the basic structure of a web page. It sets up the document head with metadata and links, and creates a body containing a root div element. The page loads a JavaScript module from a specified source file to handle dynamic content.

```
<!doctype html>

<html lang="fr">

 <head>

  <meta charset="UTF-8" />

  <link rel="icon" type="image/svg+xml" href="/vite.svg" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Vite + React</title>

 </head>

 <body>

  <!-- Root container where the React application will be mounted -->

  <div id="root"></div>

  <!-- Entry point for the JavaScript application -->

  <script type="module" src="/src/main.jsx"></script>

 </body>

</html>
```

App.module.css

Found in folder: **/src**

This CSS code defines styles for a web application’s layout and header. It ensures the main container fills at least the full viewport height and creates a fixed-height header with a bottom border. The header contains a site name styled with specific typography and color.

```
.app {

 min-height: 100vh; /* Ensures the app container takes at least the full viewport height */
```

```
}
.siteHeader {
  background: #ffffff;
  border-bottom: 1px solid #e5e7eb; /* Adds a subtle separator line below the header */
  padding: 0 1.5rem;
  height: 56px; /* Sets a fixed height for the header */
  display: flex;
  align-items: center; /* Vertically centers content inside the header */
}
.siteName {
  font-size: 1.1rem;
  font-weight: 800;
  color: #2563eb; /* Primary brand color for the site name */
  letter-spacing: -0.02em; /* Slightly tightens letter spacing for a modern look */
}
```

index.css

Found in folder: **/src**

This CSS code sets global styling rules for all elements and the body. It applies border-box sizing to every element and its pseudo-elements. It also defines default styles for the body, including removing margins, setting a font stack, background color, text color, and line height.

```
*,
*::before,
*::after {
  /* Use border-box model for all elements, making width/height include padding and border */
  box-sizing: border-box;
```

```
}

body {
  margin: 0; /* Remove default browser margin */
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif; /* Modern system font stack */
  background-color: #f4f5f7; /* Light gray background */
  color: #1a1a2e; /* Dark blue-gray text color */
  line-height: 1.6; /* Improved readability with increased line spacing */
}
```

ArticleCard.module.css

Found in folder: **/src/components**

This CSS defines styles for a card component and its child elements. It sets up a visually distinct card with a hover effect that lifts and adds shadow. The card contains a title, metadata, summary text, and a button, each styled for hierarchy and interaction.

```
.card {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 10px;
  padding: 1.5rem;
  cursor: pointer; /* Indicates the card is clickable */
  display: flex;
  flex-direction: column;
  gap: 0.5rem; /* Spacing between child elements */
  transition: transform 0.18s ease, box-shadow 0.18s ease;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.06);
}

.card:hover {
```

```
  transform: translateY(-3px); /* Lifts card on hover */
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.1);
}
.title {
  margin: 0;
  font-size: 1.15rem;
  font-weight: 700;
  color: #1a1a2e;
  line-height: 1.35;
}
.meta {
  margin: 0;
  font-size: 0.8rem;
  color: #9ca3af;
  letter-spacing: 0.02em;
}
.summary {
  margin: 0;
  font-size: 0.95rem;
  color: #4b5563;
  flex: 1; /* Allows summary to grow and fill available space */
}
.button {
  align-self: flex-start;
  margin-top: 0.5rem;
  padding: 0.45rem 1rem;
```

```
  background: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  font-size: 0.85rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.15s ease;
}
.button:hover {
  background: #1d4ed8;
}
```

ArticleDetail.module.css

Found in folder: **/src/components**

This CSS file defines styles for a container, a back button, and an article layout including its title, metadata, and content paragraphs. It sets visual properties like colors, spacing, borders, and hover effects to create a clean, responsive design. The styles ensure consistent typography and interactive feedback for the user interface.

```
.container {
  max-width: 720px;        /* Center content with a maximum width */
  margin: 0 auto;          /* Horizontal centering */
  padding: 2rem 1.5rem;    /* Internal spacing */
}
.backButton {
  background: none;
  border: 1px solid #d1d5db;
  border-radius: 6px;
```

```
  padding: 0.4rem 0.85rem;
  font-size: 0.875rem;
  color: #4b5563;
  cursor: pointer;         /* Indicates clickable element */
  transition: background 0.15s ease, color 0.15s ease; /* Smooth hover effects */
  margin-bottom: 2rem;      /* Space below button */
}
.backButton:hover {
  background: #f3f4f6;      /* Light background on hover */
  color: #1a1a2e;          /* Darker text color on hover */
}
.article {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 10px;
  padding: 2rem 2.5rem;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.06); /* Subtle shadow for depth */
}
.title {
  margin: 0 0 0.5rem 0;
  font-size: 1.75rem;
  font-weight: 800;
  line-height: 1.3;
  color: #1a1a2e;          /* Dark color for prominence */
}
.meta {
```

```
  margin: 0 0 1.75rem 0;
  font-size: 0.8rem;
  color: #9ca3af;          /* Muted color for secondary information */
  letter-spacing: 0.02em;
}
.content p {
  margin: 0 0 1.2rem 0;     /* Spacing between paragraphs */
  font-size: 1rem;
  color: #374151;
  line-height: 1.75;       /* Improved readability */
}
.content p:last-child {
  margin-bottom: 0;        /* Remove bottom margin from last paragraph */
}
```

ArticleList.module.css

Found in folder: **/src/components**

This CSS defines styles for a layout container, header, and responsive grid. It sets up a centered content section with a header containing aligned title elements. The grid creates flexible columns that automatically adjust based on available space.

```
.section {
  max-width: 1000px;       /* Constrain width for large screens */
  margin: 0 auto;          /* Center horizontally */
  padding: 2rem 1.5rem;    /* Internal spacing */
}
.header {
  margin-bottom: 2rem;     /* Space below header */
  display: flex;
```

```
  align-items: baseline;    /* Align text baselines */
  gap: 1rem;              /* Spacing between child elements */
}
.heading {
  margin: 0;
  font-size: 1.75rem;
  font-weight: 800;
  color: #1a1a2e;         /* Dark primary color */
}
.subheading {
  margin: 0;
  font-size: 0.875rem;
  color: #9ca3af;         /* Muted secondary color */
}
.grid {
  display: grid;
  /* Responsive columns: fill available space, minimum 300px each */
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 1.25rem;           /* Spacing between grid items */
}
```

articles.js

Found in folder: **/src/data**

This code defines an array of article objects, each containing metadata and content for a blog post. It exports the array as the default export for use in other parts of the application.

```
const articles = [
  {
```

id: 1,

title: "How the James Webb Space Telescope Works",

summary:

"A look at the mirror design, infrared imaging, and what JWST has already revealed about the early universe.",

content: `The James Webb Space Telescope, launched December 25, 2021, is the most powerful space observatory ever built. At its heart is a 6.5-meter primary mirror composed **of** 18 hexagonal gold-plated beryllium segments — more than six times the light-collecting area **of** Hubble. That gold coating is deliberate: it maximizes reflectivity at infrared wavelengths, where Webb does most **of** its work.

Unlike Hubble, which observes primarily **in** visible and ultraviolet light, Webb operates across near- and mid-infrared wavelengths **(**0.6 to 28 micrometers**)**. Infrared light passes through dust clouds that block visible light, letting Webb peer inside stellar nurseries and the cores **of** galaxies. It also captures the heavily redshifted light **of** the most distant galaxies — objects so far away that their light has been traveling **for** over 13 billion years, stretched from ultraviolet into the infrared by the expansion **of** the universe.

To detect **this** faint heat radiation, Webb's instruments must be kept extraordinarily cold. A five-layer sunshield the size of a tennis court blocks heat from the Sun, Earth, and Moon, holding the telescope at around −233°C (−387°F).

Since beginning science operations in 2022, Webb has delivered remarkable results: the deepest infrared view of the universe, the first direct detection of carbon dioxide in an exoplanet's atmosphere, and images **of** galaxies that formed within 300 million years **of** the Big Bang — reshaping our understanding **of** early cosmic history.` ,

author: "Editorial Team",

date: "2025-11-15",

},

{

id: 2,

title: "Why Sleep Matters for Learning and Memory",

summary:

"What happens in your brain overnight, and why skipping sleep quietly undoes the studying you did that day.",

content: `Sleep is not passive downtime — it is when the brain does much **of** its most important work. During the hours you sleep, neural circuits that fired during the day are replayed and reorganized, converting fragile short-term impressions into durable long-term memories.

The process depends on sleep's distinct stages. During non-REM slow-wave sleep — the deep, dreamless stages — the hippocampus, the brain's short-term memory hub, systematically reactivates the day's experiences and transfers them to the neocortex for permanent storage. Characteristic bursts of neural activity called sleep spindles help bind new information into existing knowledge networks.

REM sleep, which intensifies during the later cycles of the night, serves a different but complementary role. Brain imaging shows the prefrontal cortex goes quiet while emotional and associative regions remain highly active — the conditions for creative insight, flexible thinking, and the processing of emotionally charged memories. Cutting a night short preferentially reduces REM, which is why even moderate sleep restriction blunts creative problem-solving the next day.

The practical implications are clear. Students who sleep after studying recall significantly more than those who stay awake for the same period. A 90-minute nap taken between a learning session and a test can nearly double retention. Chronic sleep deprivation gradually shrinks the hippocampus and impairs the formation of new memories. The research is unambiguous: for learning to stick, sleep is not optional — it is part of the process.`,

author: "Editorial Team",

date: "2025-12-02",

},

{

id: 3,

title: "How Black Holes Are Formed",

summary:

"How dying stars, cosmic mergers, and the early universe can create some of the densest objects in space.",

content: `Black holes form when gravity compresses enough matter into a region so dense that not even light can escape. The best understood route begins with a very massive star. For most of its life, the star holds itself up by fusing lighter elements into heavier ones, releasing energy that pushes outward against gravity. Eventually, the core runs out of useful fuel. If the remaining core is massive enough, gravity wins: the core collapses while the outer layers may blast away in a supernova.

What remains depends on the mass of the core. Less extreme collapses can leave behind a neutron star, an object only about the size of a city. But if the core is heavier, it keeps shrinking until it forms a stellar-mass black hole. NASA estimates that stars born with about 20 times the Sun's mass or more can **end this** way.

Black holes can also grow after they form. They may pull **in** gas from nearby stars, swallow other compact objects, or merge with other black holes. These mergers release gravitational waves, ripples **in** space-time that observatories on Earth can detect.

The largest black holes, found **in** the centers **of** galaxies, are harder to explain. They may have grown from smaller black holes, formed through repeated mergers, or begun as massive "seeds" from collapsing gas clouds **in** the young universe. Scientists are still testing these ideas.`,

author: "Editorial Team",

date: "2026-06-10",

},

{

id: 4,

title: "The Science of Habit Formation",

summary:

"Why repeated actions become automatic, and how cues, rewards, and context shape everyday behavior.",

content: `Habits are the brain's way **of** saving effort. Instead **of** deciding from scratch every time you brush your teeth, check your phone, or make coffee, your brain learns to link a situation with an action. Psychologists call **this** a context-response association: a cue such as a time, place, emotion, or preceding action starts a behavior with little conscious planning.

At first, a **new** habit usually needs intention. You decide to stretch after waking up or read before bed. Repetition **in** the same context gradually makes the action easier to trigger. Rewards help because they tell the brain the behavior was worth repeating. The reward does not have to be dramatic; a sense **of** relief, pleasure, progress, or reduced friction can strengthen the pattern.

Neuroscience points to the basal ganglia, a group **of** deep brain structures involved **in** learning routines, selecting actions, and processing reward. As a habit strengthens, control can shift away from deliberate, goal-focused thinking and toward faster automatic responses. That is why habits can be helpful when they support goals, but stubborn when they **do** not.

There is no universal "21-day" rule. **In** one real-world study, automaticity often took weeks to months and varied widely by person and behavior. The practical lesson is simple: make the desired action easy, repeat it **in** a stable context, and attach it to a reward you actually feel.` ,

  author: "Editorial Team",

  date: "2026-06-11",

 },

];

// Exports the array of article objects as the default module export

export default articles;